

BLUESTOCK MUTUAL FUND ANALYTICS

Institutional Research & Engineering Framework

Final Report

Sayon Manna

Data Analyst Intern

Bluestock Capstone Workspace Team (MJ286)

1. Executive Summary

Bluestock Mutual Fund Analytics is positioned as a decision-grade mutual fund intelligence stack built for Indian wealth-management workflows. The platform's primary business objective is to unify fragmented fund-master data, daily NAV observations, investor transaction trails, portfolio holdings, and benchmark market series into a coherent analytical environment that supports executive oversight, portfolio evaluation, and investor-behavior diagnosis. Rather than treating mutual fund analysis as a set of disconnected charts, the framework turns the source data into a governed research asset that can be refreshed, queried, scored, and presented through a reproducible Python delivery layer.

The analytical value of the platform comes from its ability to move beyond trailing-return narratives and into institutional risk assessment. A fund may appear strong on headline growth yet still exhibit elevated downside tail exposure, unstable rolling risk-adjusted returns, excessive sector concentration, or weak investor-flow durability. The project therefore evaluates 40 schemes through a multidimensional lens: daily return continuity, annualized CAGR, Sharpe and Sortino ratios, alpha and beta against benchmark indices, historical VaR and CVaR, drawdown dynamics, and HHI-based structural concentration analysis. This framework allows decision-makers to interpret performance with the same rigor expected in buy-side or wealth-tech research environments.

The investor-side analytics are equally important. Systematic Investment Plan inflows, folio growth, transaction cadence, and cohort-level capital behavior indicate whether retail participation is broadening, whether cash-flow patterns are stable, and whether investors are clustering into a small set of preferred categories or fund houses. In practical terms, the report should show that capital deployment is not merely a volume metric; it is a behavioral signal that speaks to investor confidence, recurring contribution discipline, and product-market fit.

The final strategic advantage is the replacement of static reporting with an interactive Python Streamlit interface. A PDF-only workflow can summarize results, but it cannot support live filtering, scheme-level drill-down, page-by-page comparison, or context-sensitive exploration. The Streamlit delivery layer transforms the platform into an executive analytical cockpit where leadership can pivot by fund house, category, risk profile, state, and transaction type without rewriting queries. This is the key transition from reporting to decision support.

Editorial objective: write this section like an investment research memo, not a project synopsis. Use language such as "tail-risk boundary," "risk-adjusted efficiency," "continuity-safe daily series," "structural concentration," and "retail capital deployment patterns."

Copy-Ready Paragraph

Bluestock Mutual Fund Analytics demonstrates how a local, reproducible Python-based stack can transform fragmented mutual fund source files into an institutional analytics platform. By combining ETL governance, continuity-safe NAV engineering, quantitative risk measurement, behavioral segmentation, and an interactive Streamlit delivery layer, the project enables a richer evaluation of fund performance, investor behavior, and portfolio concentration than static reports can provide. The resulting system is suitable for executive review, research analysis, and future extension into production-grade wealth-management workflows.

2. Data Architecture & Ingestion Design

The data architecture is intentionally simple in physical storage but rigorous in analytical structure. The pipeline begins with raw CSV source feeds and ends with a relational SQLite store that supports both operational validation and downstream quantitative analysis. The raw source landscape includes fund master data, historical NAV records, AUM by fund house, SIP inflows, category inflows, industry folio counts, scheme performance metrics, investor transaction records, portfolio holdings, and benchmark index series. Each asset is treated as an immutable input, validated independently, and transformed only after schema conformity and business-rule checks have passed.

The ingestion layer should be described as a deterministic file-based workflow. First, pandas loads each source file from a path resolved relative to the project root. Then the data is checked for missing headers, data-type drift, duplicate rows, invalid monetary values, and malformed dates. Once validated, the cleaned tables are migrated into SQLite, which acts as a continuity-safe local star-schema store. This architecture is lightweight enough for a capstone environment but still disciplined enough to reflect real data-engineering practice.

The relational model is centered on `dim_fund`, with `fact_nav` and `fact_transactions` as the two core fact tables. In a conceptual extension, `dim_investor` can be treated as a logical dimension derived from investor transaction history, while `fact_performance` can be modeled as a derived analytical mart or materialized scorecard layer. The critical join key is `amfi_code`, which anchors scheme identity across the full data stack. Time-series analysis is driven by `nav_date`, while transaction behavior is driven by `transaction_date`.

ASCII Lineage Diagram

```
[Raw CSV Data Feeds]
|-- 01_fund_master.csv
|-- 02_nav_history.csv
|-- 03_aum_by_fund_house.csv
|-- 04_monthly_sip_inflows.csv
|-- 05_category_inflows.csv
|-- 06_industry_folio_count.csv
|-- 07_scheme_performance.csv
|-- 08_investor_transactions.csv
|-- 09_portfolio_holdings.csv
|-- 10_benchmark_indices.csv
|
v
[Pandas Validation & Ingestion Layer]
|-- schema validation
|-- date parsing
|-- numeric coercion
|-- duplicate handling
|-- continuity checks
|-- business-rule filtering
|
v
[SQLite Relational Star-Schema Store]
|-- dim_fund
|-- fact_nav
```

```

|-- fact_transactions
|-- fact_performance (derived mart / scorecard view)
|
v
[Streamlit Interactive Application UI]
|-- Industry Overview
|-- Fund Performance Dynamics
|-- Investor Analytics
|-- SIP and Macro Trends

```

Cross-platform Path Isolation

All file locations are resolved through `pathlib.Path`, anchored relative to the repository root rather than hard-coded to a local user directory. This is a core portability control because it keeps the same code runnable on macOS, Windows, and Linux without editing source paths. It also improves reproducibility in notebooks, scripts, and terminal execution because each module can derive the same project root from its execution context.

Production-Ready Python Code Block

```

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parents[1]
RAW_DIR = BASE_DIR / "data" / "raw"
PROCESSED_DIR = BASE_DIR / "data" / "processed"
DB_DIR = BASE_DIR / "data" / "db"
REPORTS_DIR = BASE_DIR / "reports"
SQL_DIR = BASE_DIR / "sql"

for directory in (RAW_DIR, PROCESSED_DIR, DB_DIR, REPORTS_DIR):
    directory.mkdir(parents=True, exist_ok=True)

DB_PATH = DB_DIR / "bluestock_mf.db"
SCHEMA_PATH = SQL_DIR / "schema.sql"

print(f"Project root: {BASE_DIR}")
print(f"Raw data directory: {RAW_DIR}")
print(f"Processed data directory: {PROCESSED_DIR}")
print(f"SQLite database path: {DB_PATH}")

```

Copy-Ready Paragraph

The Bluestock architecture uses a reproducible data pipeline that begins with raw CSV assets and ends with a relational SQLite analytics store. Source files are validated, normalized, and transformed using Python before being loaded into a compact star schema centered on `amfi_code`. The design intentionally isolates filesystem paths through `pathlib.Path`, which makes the workflow portable across environments and prevents hard-coded dependency on any developer's local machine layout.

3. Data Cleaning & Financial Continuity Engineering

Financial continuity engineering is one of the most important design decisions in the entire platform. Mutual fund NAV series do not update on weekends or market holidays, which means raw observations contain structural gaps that are not data errors but market-calendar effects. If those gaps are not handled consistently, any daily return series will be distorted, and downstream metrics such as trailing performance, rolling volatility, Sharpe ratio, max drawdown, and alpha-beta regression will inherit systematic bias.

The correct approach is to reconstruct a continuous daily calendar for each scheme, reindex the NAV series onto that calendar, and forward-fill the most recent valid NAV across non-trading intervals. This retains the economic meaning of the series while preserving daily continuity for analysis. The logic should be described as a "continuity-safe approximation" rather than a naive interpolation. It reflects the fact that fund NAVs are observed on every business day but remain economically valid across weekends and holidays through the carry-forward of the last trading-day value.

Transaction sanitization should be described separately from price continuity. The investor transaction file must be normalized into a consistent transaction taxonomy, with values restricted to canonical labels such as SIP, Lumpsum, and Redemption. Monetary values must be positive, timestamps must parse cleanly, and malformed or incomplete rows must be removed. Geographic descriptors such as state and city tier should be preserved because they later feed investor segmentation, capital concentration, and demographic analysis.

Skipping NAV gap-filling is not a cosmetic issue. It changes the shape of the return distribution, creates artificial zero-return gaps, compresses trailing sample windows, and distorts the denominator of risk-adjusted metrics. A 90-day rolling Sharpe ratio is only interpretable if the 90-day window is truly continuous in market time. Likewise, a 3-year CAGR should be calculated from a logically preserved return path, not from a broken series with omitted non-trading intervals.

Production-Ready Python: Continuity-Safe NAV Gap Filling

```

from __future__ import annotations

from pathlib import Path

import pandas as pd

BASE_DIR = Path(__file__).resolve().parents[1]
RAW_DIR = BASE_DIR / "data" / "raw"
PROCESSED_DIR = BASE_DIR / "data" / "processed"

def clean_nav_history() -> pd.DataFrame:
    """Load raw NAV data and forward-fill weekend and holiday gaps."""
    PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

    nav_path = RAW_DIR / "02_nav_history.csv"
    nav_df = pd.read_csv(nav_path)

    nav_df["date"] = pd.to_datetime(nav_df["date"], errors="coerce")
    nav_df = nav_df.dropna(subset=["date", "amfi_code", "nav"])
    nav_df = nav_df.drop_duplicates()
    nav_df = nav_df[nav_df["nav"] > 0]
    nav_df = nav_df.sort_values(["amfi_code", "date"])

    def _reindex_and_fill(group: pd.DataFrame) -> pd.DataFrame:
        amfi_code = group.name
        group = group.sort_values("date").copy()
        daily_index = pd.date_range(
            start=group["date"].min(),
            end=group["date"].max(),
            freq="D",
        )
        group = group.set_index("date").reindex(daily_index).ffill()
        group["amfi_code"] = amfi_code
        group = group.reset_index().rename(columns={"index": "date"})
        return group

    nav_df = (
        nav_df.groupby("amfi_code", group_keys=False)
        .apply(_reindex_and_fill)
        .reset_index(drop=True)
    )

    nav_df = nav_df[nav_df["nav"] > 0]
    nav_df = nav_df.sort_values(["amfi_code", "date"])

    output_path = PROCESSED_DIR / "clean_02_nav_history.csv"
    nav_df.to_csv(output_path, index=False)
    print(f"Saved cleaned NAV history to {output_path}")
    return nav_df

```

Transaction Sanitization Code Block

```
def clean_investor_transactions() -> pd.DataFrame:
    """Standardize transaction categories and enforce positive monetary values."""
    tx_path = RAW_DIR / "08_investor_transactions.csv"
    tx_df = pd.read_csv(tx_path)

    object_cols = tx_df.select_dtypes(include=["object"]).columns
    tx_df[object_cols] = tx_df[object_cols].apply(lambda col:
col.astype(str).str.strip())

    tx_df["transaction_date"] = pd.to_datetime(tx_df["transaction_date"],
errors="coerce")
    tx_df["transaction_type"] = tx_df["transaction_type"].str.upper()

    type_map = {
        "SIP": "SIP",
        "LUMPSUM": "Lumpsum",
        "REDEMPTION": "Redemption",
    }
    tx_df["transaction_type"] = tx_df["transaction_type"].map(type_map)

    tx_df = tx_df.dropna(subset=["transaction_date", "transaction_type",
"amount_inr"])
    tx_df = tx_df[tx_df["amount_inr"] > 0]
    tx_df = tx_df[tx_df["kyc_status"].isin(["Verified", "Pending"])]

    output_path = PROCESSED_DIR / "clean_08_investor_transactions.csv"
    tx_df.to_csv(output_path, index=False)
    return tx_df
```

Copy-Ready Paragraph

Financial continuity engineering is a non-negotiable requirement in mutual fund analytics because NAV series are naturally discontinuous on weekends and market holidays. The project resolves this by constructing a complete daily calendar for every scheme and forward-filling the last valid NAV into non-trading intervals. This preserves daily return continuity and prevents artificial volatility spikes, mis-specified CAGR windows, and broken rolling performance metrics. In parallel, transaction cleaning enforces canonical category labeling and positive-value constraints so that capital flow analysis remains economically valid.

4. Advanced Quantitative Risk Metrics

The advanced analytics layer is the risk heart of the report. It should be written in the tone of a research desk or portfolio oversight memo. The goal is not simply to calculate statistics; it is to show how those statistics interact to reveal downside vulnerability, performance stability, and construction risk.

4.1 Value at Risk (95% Historical VaR)

Historical VaR at the 95% confidence level estimates the daily downside threshold at the 5th percentile of the empirical return distribution. In practical terms, this identifies the level of daily loss that should be exceeded only in the worst 5% of observed return days. VaR is not a forecast in the parametric sense; it is an empirical tail boundary derived from realized market behavior.

The appropriate interpretation in a mutual fund context is scheme-specific downside sensitivity. A fund with a relatively mild VaR profile is less prone to extreme daily losses, while a fund with a deeply negative VaR suggests structurally heavier tail exposure. This metric is most useful when interpreted alongside CVaR and drawdown, because VaR alone does not reveal the severity of losses beyond the cutoff.

4.2 Conditional Value at Risk (95% CVaR)

CVaR, also known as Expected Shortfall, measures the average of all return observations that fall below the VaR threshold. This is a more severe and more informative tail-risk statistic than VaR because it evaluates the average damage in the left tail rather than just the boundary point.

In the final report, CVaR should be framed as a measure of "tail loss intensity." Two funds can share the same VaR and still have very different CVaR profiles if one exhibits more catastrophic losses beyond the threshold. That makes CVaR especially important for comparing funds with similar headline volatility but different downside structure.

4.3 Rolling 90-Day Sharpe Ratio

The rolling Sharpe ratio measures risk-adjusted performance through time, not just over an entire history. The report should state that the project uses a 90-trading-day window and annualizes using 252 active market days, not 365 calendar days. This distinction matters because financial markets do not trade every calendar day, and annualization should reflect market-session normalization.

The rolling structure is useful because it reveals whether a fund's performance efficiency is stable or regime-dependent. A static Sharpe ratio may look strong over a long horizon while hiding periods of compression, drawdown clustering, or elevated short-term risk. The rolling version exposes that behavior directly.

4.4 Herfindahl-Hirschman Index (HHI)

HHI is the concentration metric used to assess portfolio diversification. The formula is the sum of squared relative allocation weights across sectors or holdings:

$$HHI = \sum (weight_i^2)$$

The higher the HHI, the more concentrated the portfolio. The lower the HHI, the more diversified the allocation base. For equity schemes, this matters because sector concentration can produce hidden style risk even when the fund appears diversified at the stock count level. A fund with many holdings can still be highly concentrated if a small number of sectors dominate the portfolio weight.

4.5 Production-Ready Python Code Blocks

Historical VaR and CVaR

```
import numpy as np
import pandas as pd

def compute_var_cvar(returns: pd.Series, alpha: float = 0.05) -> tuple[float, float]:
    """Compute 95% historical VaR and CVaR from a daily return series."""
    clean_returns = returns.dropna().astype(float)
    var_95 = clean_returns.quantile(alpha)
    cvar_95 = clean_returns[clean_returns <= var_95].mean()
    return float(var_95), float(cvar_95)

risk_rows = []

for amfi_code, group in returns_frame.groupby("amfi_code", sort=True):
    daily_returns = group["daily_return"].dropna()
    if daily_returns.empty:
        continue

    var_95, cvar_95 = compute_var_cvar(daily_returns)

    risk_rows.append(
        {
            "amfi_code": amfi_code,
            "scheme_name": group["scheme_name"].dropna().iloc[0],
            "fund_house": group["fund_house"].dropna().iloc[0],
            "category": group["category"].dropna().iloc[0],
            "observations": int(daily_returns.shape[0]),
            "var_95_daily": var_95,
            "cvar_95_daily": cvar_95,
        }
    )

var_cvar_report =
pd.DataFrame(risk_rows).sort_values("var_95_daily").reset_index(drop=True)
var_cvar_report.to_csv(PROCESSED_DIR / "var_cvar_report.csv", index=False)
```

Rolling 90-Day Sharpe Ratio

```
import numpy as np
import pandas as pd

ROLLING_WINDOW = 90
TRADING_DAYS = 252

rolling_sharpe_frames = []

for amfi_code, group in returns_frame.groupby("amfi_code", sort=True):
    group = group.sort_values("nav_date").copy()
    rolling_mean = group["daily_return"].rolling(ROLLING_WINDOW).mean()
    rolling_std = group["daily_return"].rolling(ROLLING_WINDOW).std()
    rolling_sharpe = (rolling_mean / rolling_std) * np.sqrt(TRADING_DAYS)

    rolling_sharpe_frames.append(
        pd.DataFrame(
            {
                "nav_date": group["nav_date"],
                "amfi_code": amfi_code,
                "scheme_name": group["scheme_name"],
                "rolling_sharpe_90d": rolling_sharpe,
            }
        )
    )

rolling_sharpe_df = pd.concat(rolling_sharpe_frames, ignore_index=True)
```

HHI Concentration Calculation

```
import numpy as np
import pandas as pd

sector_weights = (
    holdings_df.groupby(["amfi_code", "sector"], as_index=False)["weight_pct"]
    .sum()
)

sector_weights["sector_weight_share"] = (
    sector_weights["weight_pct"]
    / sector_weights.groupby("amfi_code")["weight_pct"].transform("sum")
)

hhi_rows = []

for amfi_code, group in sector_weights.groupby("amfi_code", sort=True):
    hhi_value =
float(np.square(group["sector_weight_share"].to_numpy(dtype=float)).sum())
    hhi_rows.append(
        {
            "amfi_code": amfi_code,
            "sector_count": int(group["sector"].nunique()),
```

```

        "hhi": hhi_value,
        "concentration_profile": "High HHI" if hhi_value >= 0.18 else "Low
Diversified HHI",
    }
)

hhi_report = pd.DataFrame(hhi_rows).sort_values("hhi",
ascending=False).reset_index(drop=True)

```

Copy-Ready Paragraph

The quantitative risk layer integrates historical tail metrics, windowed performance measures, and concentration analytics to form a multidimensional view of fund quality. Historical VaR identifies the empirical downside boundary, CVaR measures the severity of losses beyond that boundary, and rolling Sharpe reveals whether risk-adjusted efficiency is stable through time. HHI extends the analysis from returns to portfolio construction by quantifying sector concentration and exposing structural diversification risk.

5. D5 & B2 Dashboard Interface Walkthrough

The dashboard section should read like a product and controls review. It is not enough to state that pages exist; the report should specify what each page is designed to prove and what the evaluator should inspect in the export artifacts.

5.1 Page 1: Industry Overview

This page is the macro-level overview of the mutual fund industry. It should present the key institutional KPIs such as total industry AUM, monthly SIP inflows, active folios, and total schemes. Beneath the KPI row, the AUM trend line should show the evolution of industry scale over time, while the AMC dominance bar chart should rank fund houses by cumulative AUM to expose concentration across asset managers.

The evaluator should look for the visual hierarchy: KPI cards at the top, temporal AUM growth in the center, and fund-house dominance in the comparative panel. The page should be visually compact but unmistakably executive in style.

[\[Insert Asset: reports/01_industry_overview.pdf / png\]](#)

The asset should show four high-prominence KPI cards, a clean industry AUM trend line, and an AMC dominance chart with clear unit labeling. The evaluator should confirm that the page visually communicates industry scale, concentration, and growth direction at a glance.

5.2 Page 2: Fund Performance Dynamics

This page is the performance and risk workbench. The risk-return scatter plot should map annualized return on the x-axis and risk or standard deviation on the y-axis, with bubble size representing fund AUM. A sortable institutional scorecard should sit alongside the scatter plot and include fields such as scheme name, 3-year return, Sharpe ratio, Sortino ratio, alpha, beta, expense ratio, and risk grade.

The page should also expose interactive slicers for fund house, category, and plan type. The evaluator should verify that a selected filter combination updates both the scatter plot and the scorecard instantly, and that the display remains stable when a filter returns no matching rows.

[\[Insert Asset: reports/02_fund_performance.pdf / png\]](#)

The asset should show a risk-return matrix, a clean scorecard table, and responsive filters. The evaluator should confirm that high-performing funds are not only visible in the chart but also numerically defensible through the scorecard.

5.3 Page 3: Investor Demographics and Behavioral Analytics

This page converts mutual fund analysis from product-only reporting into retail behavior intelligence. It should include geographic capital inflow by state, transaction mix by product type, and age cohort ticket size distribution. Where appropriate, city tier segmentation should distinguish metro behavior from regional participation.

The evaluator should inspect whether the page tells a story about where retail capital is coming from, how it is being deployed, and whether certain age buckets are producing larger or more volatile ticket sizes. This page is important because it transforms "investor count" into actionable behavioral segmentation.

[\[Insert Asset: reports/03_investor_analytics.pdf / png\]](#)

The asset should contain a geographic ranking visual, a product mix donut chart, and a ticket-size distribution plot. The evaluator should be able to infer regional concentration, behavior by cohort, and the retail mix of SIP versus lump-sum capital.

5.4 Page 4: SIP and Macro Market Trends

This page should connect household investment behavior to market context. The primary visual should overlay monthly SIP inflows with Nifty 50 trend movement so that the evaluator can see whether recurring investment strength aligns with broader market conditions. The secondary visual should be a category inflow heatmap showing which fund categories are absorbing capital month by month.

This page should emphasize that SIP inflows are not just a sales metric; they are a macro-behavior indicator. The evaluator should look for the intersection between household discipline and market cycle behavior.

[\[Insert Asset: reports/04_sip_trends.pdf / png\]](#)

The asset should show a dual-axis SIP-versus-index view and a monthly category heatmap. The evaluator should confirm that the page links retail systematic investing to the market regime and category capital intensity.

5.5 Interface Design and Safety Controls

The report should explicitly note that the Streamlit application uses cascading multi-select filters and safety guards for empty selections. These are not cosmetic features. They are part of the UI contract because they prevent crashes, preserve user trust, and allow direct on-screen interrogation of the analytics layer without code changes.

Copy-Ready Paragraph

The Streamlit dashboard operationalizes the analytics stack by transforming charts into an interactive decision environment. Each page is designed around a distinct executive use case: industry scale, fund performance, investor behavior, and macro-linked SIP flow analysis. Cascading filters allow the evaluator to interrogate the data from multiple angles without rewriting queries, while defensive empty-filter logic ensures that the application remains stable even when a selected combination returns no matching rows.

6. System Limitations & Strategic Recommendations

A high-quality final report should acknowledge the scope of the implementation rather than overselling it. The current platform is highly effective for local analytics and presentation, but it is not yet a distributed production system. SQLite is deliberately chosen for simplicity and portability, which means it introduces single-writer limitations and is not ideal for concurrent enterprise workloads. The data pipeline also operates on periodic source files rather than live streaming feeds, so it does not yet support intraday refresh or real-time alerting. In addition, some of the current views are historical or snapshot-based, which means they are excellent for research and reporting but not yet equivalent to live market execution systems.

The strategic recommendation section should then describe a credible next-phase roadmap. The first recommendation is migration from SQLite to a managed PostgreSQL cloud cluster to support concurrency, stronger transactional guarantees, and broader application sharing. The second is workflow orchestration using a scheduler or pipeline framework so that data refresh, validation, and model updates become automated. The third is extension into predictive analytics, especially churn modeling, SIP retention scoring, category-switch prediction, and investor attrition detection. A final recommendation is to introduce a multi-agent research layer using frameworks such as CrewAI or LangGraph for automated narrative generation, exception handling, and analyst assistance.

Copy-Ready Paragraph

The current implementation is intentionally optimized for local reproducibility and analytical clarity, but it is not yet a full enterprise runtime. SQLite introduces single-writer constraints, and the platform still relies on periodic source refreshes rather than real-time market streaming. The recommended next phase is a controlled transition to PostgreSQL-backed cloud infrastructure, workflow orchestration, and predictive modeling so that the platform can evolve from a capstone analysis environment into a scalable wealth-tech decision system.

Appendix: System Initialization & Deployment Guide

A. macOS

```
cd "/Users/sayonmanna/project 2"
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python run_pipeline.py
streamlit run dashboard/app.py
```

B. Windows PowerShell

```
cd "C:\path\to\project 2"
py -3 -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
python run_pipeline.py
streamlit run dashboard/app.py
```

C. Windows Command Prompt

```
cd "C:\path\to\project 2"
py -3 -m venv venv
venv\Scripts\activate.bat
pip install -r requirements.txt
python run_pipeline.py
streamlit run dashboard/app.py
```

D. Linux

```
cd "/path/to/project 2"
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python run_pipeline.py
streamlit run dashboard/app.py
```

E. Deployment Sequence Explanation

The system should be initialized in the following order:

- Create and activate the virtual environment.
- Install dependencies from requirements.txt.
- Run the master pipeline orchestrator to clean data, rebuild the SQLite database, and validate the rules engine.
- Launch the dashboard locally in Streamlit.
- Export PDF and PNG artifacts from the browser print dialog or OS-level capture workflow if required.